다양한 도메인을 넘나드는, 카멜레온 개발자

국민대학교 소프트웨어학과

김선미

Backend Developer

SQLD TOEIC IH

DB 커넥션 점유

90% ↓

SSAJU · 500ms→50ms

커넥션 풀 타임아웃

0건

SSAJU · 정합성 훼손 없이 달성

랭킹 API 응답 개선

인덱스·쿼리 개선

수어의 달인 · 콜드 3.6초 병목 해소

2021.03

국민대학교

소프트웨어학과 입학

2026.02

국민대학교

소프트웨어학과 졸업

2026.01

SSAFY 15기

삼성 청년 SW 아카데미 입과

• 현재

SSAFY 교육 수료중

알고리즘 · 프로젝트 · 취업 지원

SKILLS

Tech Stack & Collaboration

BACKEND · LANGUAGE

Java 21 | Spring Boot | Spring Security | Spring AI | Thymeleaf | Python | FastAPI

AI TOOLS

Claude | Claude Code | AI를 개발 프로세스에 통합하는 역량

AI · DATA

OpenAI API | gpt-4o-mini

- 생성형 AI를 서비스 핵심 로직에 통합
- 프롬프트 설계부터 응답 파싱까지 전담

DATABASE

MySQL | Redis | QueryDSL

COLLABORATION

GitHub | GitLab | Jira | CodeRabbit | JMeter

INFRA · DEVOPS

Docker | GitHub Actions | AWS EC2 · RDS | Lambda · SQS · SNS | Contabo | Cloudflare

PROJECT 01

SSAJU

생성형 AI 기반 맞춤형 진로(사주) 분석 서비스

기간	팀 구성	ssaju.net
2026.04 — 서비스 운영 중	2인 (백엔드 담당)	실서비스 운영 중

주요 담당 역할 - 관리자 페이지 구축

Spring Security와 Thymeleaf 기반으로
실시간 대시보드, 유저 모니터링, API 한도 조정, 피드백 통계 기능을 갖춘
통합 어드민 시스템을 구축

주요 담당 역할 - Redis 분산락 적용

동시 요청이 같은 사주/궁합/컨설팅 결과를 중복 생성하려 할 때, 지금처럼 DB 제약 위반 예외를 잡아 흡수하는 방식 대신
Redis 분산락으로 애초에 경쟁 자체를 차단해 **레이스 컨디션을 없애기 위함**
이를 통해 예외 기반 흡수·재조회 로직과 우회용 트랜잭션 분리 코드를 단순한 조회 후 생성 흐름으로 코드 작성

Java 21 · Spring Boot 4.0.5 · **Spring AI** · Python · FastAPI · MySQL · Docker · Cloudflare · GitHub Actions



ssaju.net

SSAJU · AI 활용

SDD 도입 & 파이프라인 자동화

문제 상황

AI 활용 개발 시 프롬프트 엔지니어링에만 의존하여 산출물 품질이 일관되지 않고, 이를 검증할 수 있는 체계가 부재함

해결 — SDD 도입

Spec-Driven Development(SDD) 체계 수립: 명세서 → 구현 → 검증으로 이어지는 구조화된 개발 흐름을 확립

Git 단계 AI 자동화 파이프라인 및 CodeRabbit 도입을 통해 **'작성자 → 검증자'** 역할을 고도화

산출물 일치율

90.3%

명세 → 구현 간 정합성 검증 완료

테스트 커버리지

70%

Git 단계 자동화 파이프라인

명세 작성
SDD 문서화

→

구현 · 커밋
AI 보조 개발

→

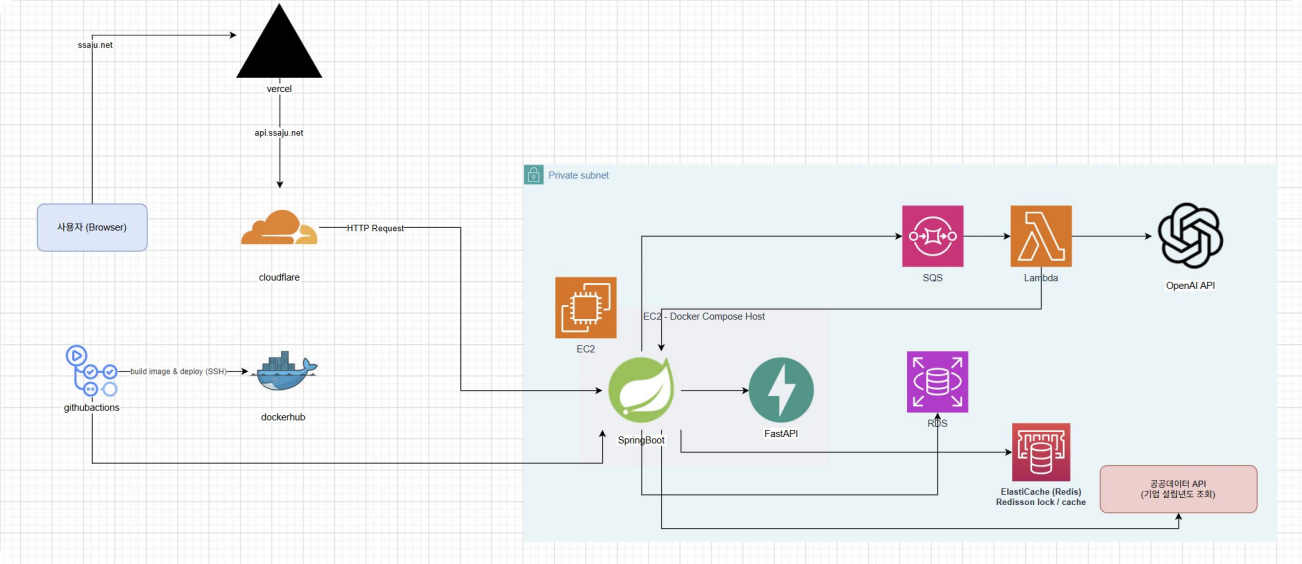
PR 생성
자동화 트리거

→

CodeRabbit 리뷰
AI 자동 검증

SSAJU · 아키텍처

비동기 아키텍처 & SQS·Lambda 도입 배경



문제 — Cloudflare 150초 타임아웃

Cloudflare 무료 요금제는 **150초 타임아웃 제한**.
SSAJU의 생성형 AI API 특성상 응답 시간이
이를 초과하면 Recover 시도조차 못한 채
사용자에게 타임아웃 발생 보고

해결 — AWS SQS 비동기 처리

AWS SQS를 도입해 AI 요청을 큐로 **비동기 처리**,
타임아웃 제약 없이 안정적으로 응답 완료

DLQ 감지 — Spring Scheduler

10분마다 스케줄러로
DLQ를 감지해 상태를 Denied로 전환,
사용자에게 오류를 안내하고 **재시도를 유도**

SSAJU · 트러블슈팅

트랜잭션 최적화 & 동시성 제어

문제 상황 ① — DB 커넥션 풀 고갈

외부 API 호출 등 무거운 I/O 작업이 하나의 큰 트랜잭션에 묶여 커넥션 점유 시간 500ms 발생 → 처리량 5건/5초 병목

1차 시도 및 한계 — Trouble

외부 API와 DB 트랜잭션을 `REQUIRES_NEW`로 분리 시도
→ 부모/자식 트랜잭션이 커넥션을 이중 점유하며 커넥션 풀 고갈 발생, 처리량 10건으로 미미한 증가

문제 상황 ② — 데이터 정합성 문제

분리된 트랜잭션은 예외 발생 시 자동 롤백이 안 되어, 외부 API 에러 시에도 쿼터가 차감되는 버그 발생

성능 돌파

Redis 분산락 도입

커넥션 점유를 DB 밖(Redis)으로 빼내 커넥션 풀 문제를 완전히 우회, 처리량 10건 → 800건/5초 (80배)

정합성 확보

보상 트랜잭션 (Saga 패턴)

DB 쿼리, API 에러가 발생한 경우, @Retryable, @Recover를 사용하여 쿼터 차감을 막고 데이터 정합성 유지 가능

DB 커넥션 점유 시간

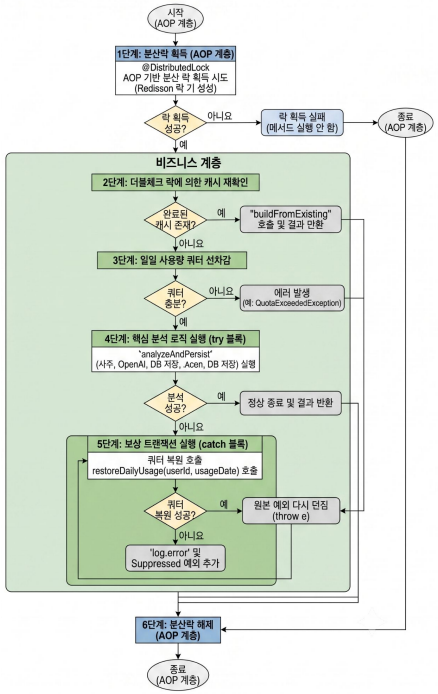
90% ↓

500ms → 50ms

동시 요청 처리 가능 건수

80배 ↑

10건 → 800건



PROJECT 02

수어의 달인

웹캠 실시간 피드백과 게임으로 배우는 지문자·기초 수어 학습 플랫폼

기간
2026.07 — 2026.08

역할
6인 팀 · 백엔드 리드

주요 담당 역할 - JPA 낙관적 락 적용

방 입장 시 두 사용자가 같은 방을 동시에 덮어쓰는 **lost update**를 막기 위해 **낙관적 락**을 도입했고,
평소엔 락 비용 없이 충돌 시에만 **409**로 감지하는 대신 **실패 시 재작업·재시도 부담을 감수**
방당 최대 2명이라는 낮은 경합 빈도상 상시 대기 비용이 드는 비관적 락보다 이 방식이 유리하다고 판단

주요 담당 역할 - k6 테스트 시나리오 작성 및 측정

k6 시나리오(방 생성/입장/나가기 churn, 로비 SSE 팬아웃)와는 별도로 GET /rankings를 **10만 유저 규모**로
직접 부하 측정하는 시나리오 측정, 그 결과 DB 스캔은 0.48초인데 API 응답은
웜 1.4~1.5초·콜드 3.6초까지 벌어지는 병목을 랭킹 조회에서 잡아내 **인덱스·쿼리 구조 개선으로 해결**

Spring Boot | Spring Security | Spring Data JPA | WebSocket | SSE | WebRTC | MySQL | AWS EC2



수어의 달인

수어의 달인 · 트러블슈팅

출석체크 동시성 문제와 격리 수준 튜닝

PROBLEM

다중 스레드 출석체크 시 UNIQUE 제약 위반 에러 및 순환 참조 롤백 발생

CAUSE

REPEATABLE READ 특성상 대기 후 깨어난 스레드가 선행 트랜잭션의 커밋을 보지 못함

SOLUTION

트랜잭션 격리 수준을 **READ_COMMITTED**로 하향 조정

RESULT

에러 원천 차단,
복구 로직 없이 단일 메서드로 정리
코드 길이 및 복잡한 Self 참조 등 제거

Before —

```
public AttendanceService(
        AttendanceRepository attendanceRepository,
        GrowthRewardService growthRewardService,
        GrowthPolicyProperties policy,
        Clock growthClock,
        @Lazy AttendanceService self) {
    this.attendanceRepository = attendanceRepository;
    this.growthRewardService = growthRewardService;
    this.policy = policy;
    this.growthClock = growthClock;
    // this.foo(...)로는 프록시를 안 거쳐 새 트랜잭션이 안 열리므로
    // 지연 주입된 자기 자신을 거친다.
    this.self = self;
}

// @Transactional이 아님 — 실패 시 새 트랜잭션에서 재조회하기 위해
// attemptCheckIn / resolveExistingCheckIn 두 단계로 분리해야 했다.
public AttendanceCompletionResponse checkIn(Long userId) {
    try {
        return self.attemptCheckIn(userId);
    } catch (DataIntegrityViolationException e) {
        return self.resolveExistingCheckIn(userId);
    }
}

@Transactional
public AttendanceCompletionResponse attemptCheckIn(Long userId) {
    LocalDate serviceDate = LocalDate.now(growthClock);
    UserPet pet = growthRewardService.lockPet(userId);

    Optional<Attendance> existing =
            attendanceRepository.findByUserIdAndAttendanceDate(userId, serviceDate);
    if (existing.isPresent()) {
        return completion(existing.get(), false, 0, pet);
    }

    int streak = attendanceRepository.findTopByUserIdOrderByAttendanceDateDesc(userId)
            .filter(last -> last.getAttendanceDate().equals(serviceDate.minusDays(1)))
            .map(last -> last.getStreakCount() + 1)
            .orElse(1);
    Attendance attendance = new Attendance(userId, serviceDate, streak);
    // REPEATABLE READ 스냅샷 때문에 existing.isPresent()가 낡은 답("없음")을 줄 수 있다.
    // 천재 안전장치는 UNIQUE 제약이고, 위반 시 예외를 그대로 던져 롤백시킨다.
```

After —

```
@Service
@RequiredArgsConstructor
public class AttendanceService {

    private final AttendanceRepository attendanceRepository;
    private final GrowthRewardService growthRewardService;
    private final GrowthPolicyProperties policy;
    private final Clock growthClock;

    @Transactional(isolation = Isolation.READ_COMMITTED)
    public AttendanceCompletionResponse checkIn(Long userId) {
        LocalDate serviceDate = LocalDate.now(growthClock);
        UserPet pet = growthRewardService.lockPet(userId);

        Optional<Attendance> existing =
                attendanceRepository.findByUserIdAndAttendanceDate(userId, serviceDate);
        if (existing.isPresent()) {
            return completion(existing.get(), false, 0, pet);
        }

        int streak = attendanceRepository.findTopByUserIdOrderByAttendanceDateDesc(userId)
                .filter(last -> last.getAttendanceDate().equals(serviceDate.minusDays(1)))
                .map(last -> last.getStreakCount() + 1)
                .orElse(1);
        Attendance attendance = new Attendance(userId, serviceDate, streak);
        attendanceRepository.save(attendance);
        growthRewardService.rewardLocked(pet, policy.getAttendanceExp());
        return completion(attendance, true, policy.getAttendanceExp(), pet);
    }
```

수어의 달인

도입보다 어려운 정착 — SOP로 실행 장벽 제거

문제 상황

4주라는 짧은 기간과 팀원들의 개발 프로세스 경험 부족으로 Git·Jira 교육에 시간을 쏟기 어려웠고, 지정된 Git 컨벤션조차 지켜지지 않는 상황

배포 문서

팀 내 공유, 표준 작업 절차(SOP) 마련 팀원들이 개발 외의 시간을 최대한 줄여 비즈니스 로직 개발에 힘을 쏟도록 함

효과

초기 적응 부담 감소, AI 워크플로우 조기 정착 팀원들이 Commit Convention, PR 컨벤션에 묶여있지 않아 더욱 편하게 개발 가능

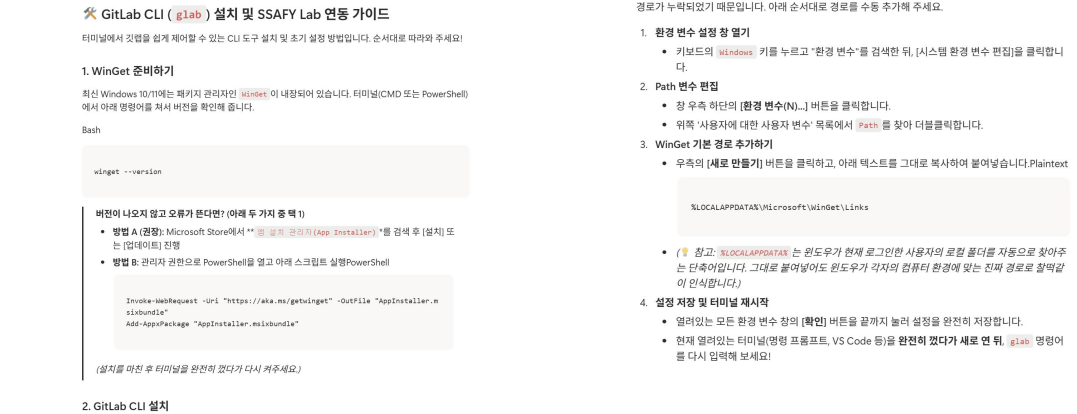
SOP 문서 주요 구성 요소

01 **GitWorkFlow 지침**
AI가 지켜야 할 MR/Commit 규칙 정의

02 **glab CLI 설치 & 인증 가이드**
GitLab API 연동 및 CLI 환경 세팅

03 **표준 작업 절차 (SOP)**
기능 개발 → 이슈 추적 → 테스트 → PR 자동화

SOP 문서 — Git Workflow · glab 설치 가이드



🛠 GitLab CLI (`glab`) 설치 및 SSAFY Lab 연동 가이드

터미널에서 깃랩을 쉽게 제어할 수 있는 CLI 도구 설치 및 초기 설정 방법입니다. 순서대로 따라와 주세요!

1. WinGet 준비하기

최신 Windows 10/11에는 패키지 관리자인 `WinGet` 이 내장되어 있습니다. 터미널(CMD 또는 PowerShell)에서 아래 명령어를 쳐서 버전을 확인해 줍니다.

Bash

```
winget --version
```

버전이 나오지 않고 오류가 뜬다면? (아래 두 가지 중 택 1)

- **방법 A (권장):** Microsoft Store에서 "`앱 설치 관리자(App Installer)`"를 검색 후 [설치] 또는 [업데이트] 진행
- **방법 B:** 관리자 권한으로 PowerShell을 열고 아래 스크립트 실행PowerShell

```
Invoke-WebRequest -Uri "https://aka.ms/getwinget" -OutFile "AppInstaller.msixbundle"
Add-AppxPackage "AppInstaller.msixbundle"
```

(설치를 마친 후 터미널을 완전히 껐다가 다시 켜주세요.)

2. GitLab CLI 설치

🚨 트러블슈팅: 터미널을 껐다 켰는데도 `glab` 명령어를 찾을 수 없을 때

WinGet으로 설치를 마쳤음에도 `glab` 명령어가 작동하지 않는다면, 윈도우 환경 변수에 WinGet의 기본 경로가 누락되었기 때문입니다. 아래 순서대로 경로를 수동 추가해 주세요.

1. **환경 변수 설정 창 열기**
   - 키보드의 `Windows` 키를 누르고 "환경 변수"를 검색한 뒤, [시스템 환경 변수 편집]을 클릭합니다.
2. **Path 변수 편집**
   - 창 우측 하단의 **[환경 변수(N)...]** 버튼을 클릭합니다.
   - 위쪽 '사용자에 대한 사용자 변수' 목록에서 `Path` 를 찾아 더블클릭합니다.
3. **WinGet 기본 경로 추가하기**
   - 우측의 **[새로 만들기]** 버튼을 클릭하고, 아래 텍스트를 그대로 복사하여 붙여넣습니다.Plaintext

```
%LOCALAPPDATA%\Microsoft\WinGet\Links
```

   - *(💡 참고: `%LOCALAPPDATA%` 는 윈도우가 현재 로그인한 사용자의 로컬 폴더를 자동으로 찾아주는 단축어입니다. 그대로 붙여넣어도 윈도우가 각자의 컴퓨터 환경에 맞는 진짜 경로로 찰떡같이 인식합니다.)*
4. **설정 저장 및 터미널 재시작**
   - 열려있는 모든 환경 변수 창의 **[확인]** 버튼을 끝까지 눌러 설정을 완전히 저장합니다.
   - 현재 열려있는 터미널(명령 프롬프트, VS Code 등)을 **완전히 껐다가 새로 연 뒤**, `glab` 명령어를 다시 입력해 보세요!

수어의 달인

이슈부터 PR까지, 맥락 단절 없는 End-to-End

Jira 티켓(MCP) → **AI Agent** → Git Commit 자동 작성 → PR 자동 작성 → 리뷰

문제 정의

이슈 관리(Jira, GitHub 등)와 IDE를 번갈아 확인하며 발생하는 Context Switching 비용 및 개발 몰입도(Flow) 저하 또한 반복적인 Commit, PR Message 작성으로 인한 피로도 증가

동작 방식

MCP로 Jira 티켓의 요구사항을 그대로 읽어 커밋 메시지와 PR 본문에 반영, 이슈 번호–커밋–PR이 한 맥락으로 연결

핵심 성과

지라 티켓 읽기, 코드 작성, ADD, Commit, Push, PR 작성의 다단계를 AI Agent에게 프롬프트 입력 한 번으로 PR 작성까지 가능하도록 변경

파이프라인 실행 결과

ci: backend 운영 브랜치 배포 규칙 적용

정승욱 authored Jul 22, 2026

실제 사람이 작성한 Commit, 오타가 있고 설명이 명확하지 않은 부분

↓

feat: 공통 타임스탬프/예외 처리 인프라 추가

김선미 authored Jul 21, 2026

```
- BaseTimeEntity(createdAt/updatedAt, JPA Auditing) 추가, SuhwaApplication에 @EnableJpaAuditing 등록 - BusinessException + ErrorCode(HTTP 상태 매핑) + GlobalExceptionHandler로 서비스 계층 예외를 계약(contracts/*.yaml)된 응답 형식(code/message)으로 일원화
```

AI Agent가 Commit 제목부터 내용까지 작성, 더 상세한 모습

CONTACT

김선미

Backend Developer

Email
kpdipgo2@gmail.com

GitHub
github.com/SUNMI-KIM

Blog
velog.io/@hariaus

다양한 도메인을 넘나들며, 끊임없이 배우는 개발자 김선미입니다.